

INTRODUCTION & REVIEW INTERNET PROTOCOL AND SERVICES

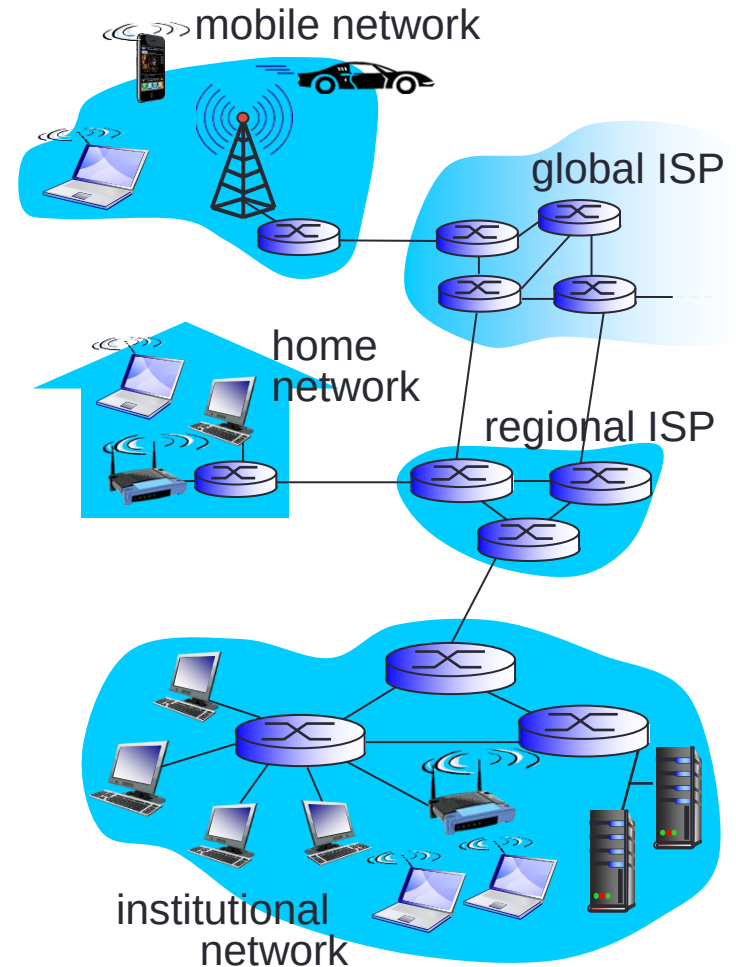
Some slides have been taken from: *Computer Networking: A Top Down Approach Featuring the Internet*, 3rd edition. Jim Kurose, Keith Ross. Addison-Wesley, July 2004. All material copyright 1996-2004. J.F Kurose and K.W. Ross, All Rights Reserved.

Contents

- Internet protocol stack
- Application layer
- UDP & TCP
- Network layer

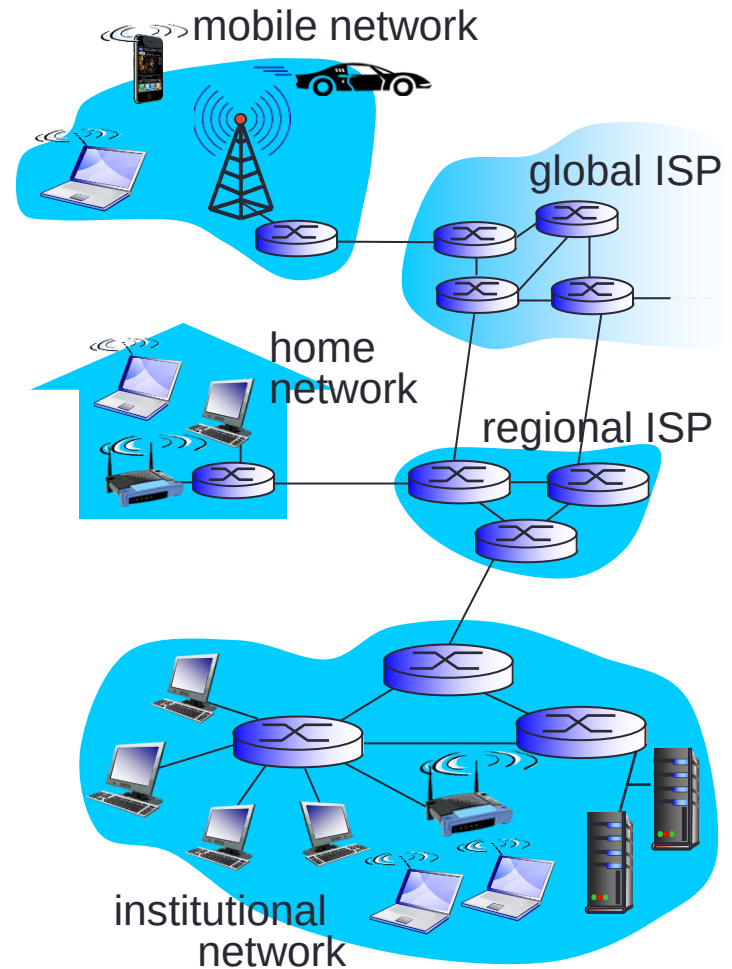
What's the Internet?

- *Internet*: “network of networks”
 - Interconnected ISPs
- *protocols* control sending, receiving of msgs
 - e.g., TCP, IP, HTTP, Skype, 802.11
- *Internet standards*
 - RFC: Request for comments
 - IETF: Internet Engineering Task Force



What's the Internet?

- *Infrastructure that provides services to applications:*
 - Web, VoIP, email, games, e-commerce, social nets, ...
- *provides programming interface to apps*
 - hooks that allow sending and receiving app programs to “connect” to Internet
 - provides service options, analogous to postal service



What's a protocol?

human protocols:

- “what's the time?”
- “I have a question”
- introductions

... specific msgs sent

... specific actions taken
when msgs received,
or other events

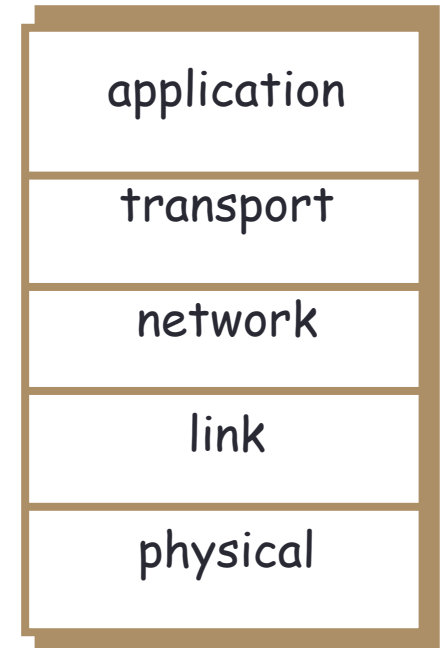
network protocols:

- machines rather than humans
- all communication activity in Internet governed by protocols

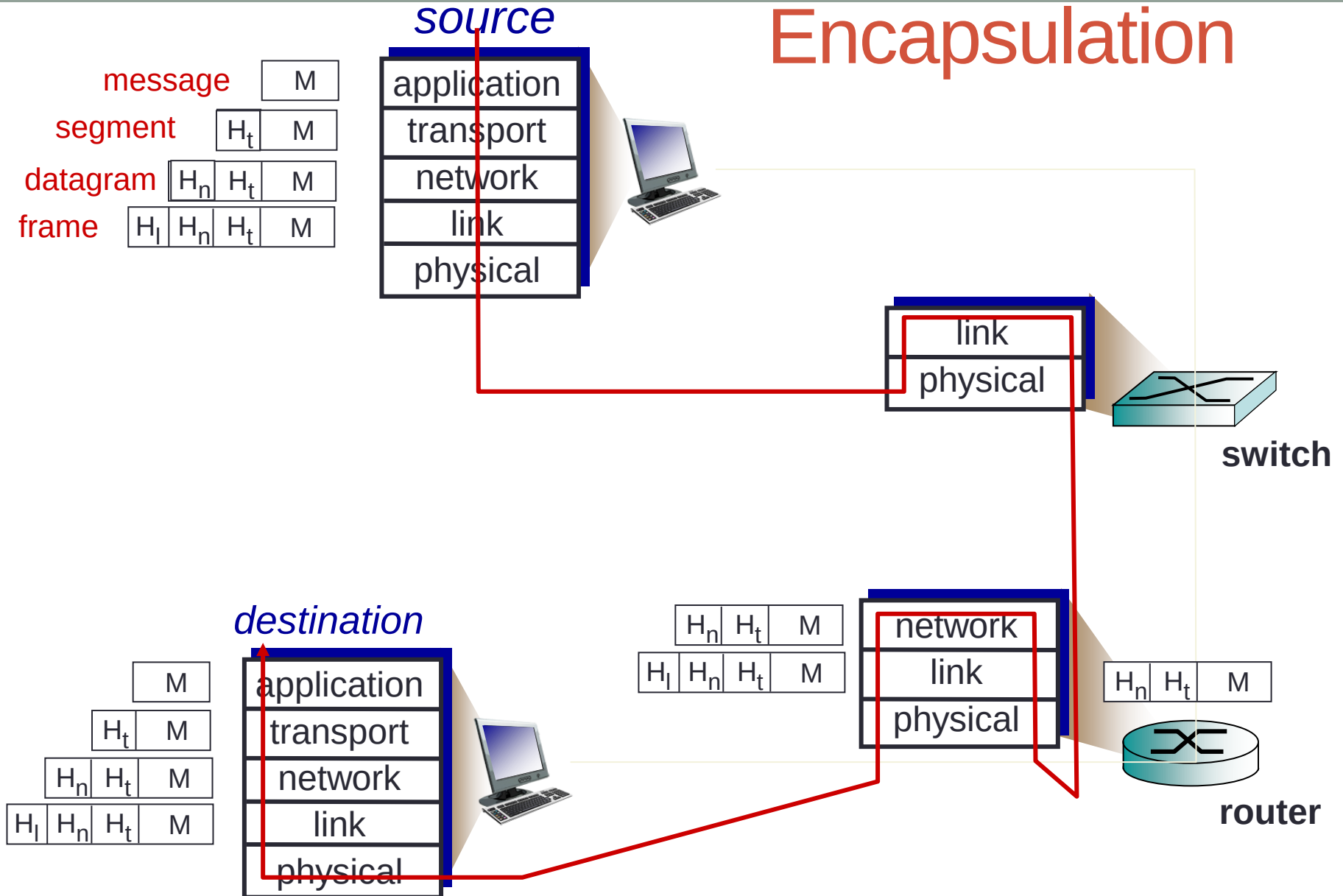
protocols define format, order of msgs sent and received among network entities, and actions taken on msg transmission, receipt

TCP/IP protocol stack

- **application:** supporting network applications
 - FTP, SMTP, STTP
- **transport:** host-host data transfer
 - TCP, UDP
- **network:** routing of datagrams from source to destination
 - IP, routing protocols
- **link:** data transfer between neighboring network elements
 - PPP, Ethernet
- **physical:** bits “on the wire”



Encapsulation



Application layer

- E-mail
- Web
- Instant messaging
- Remote login
- P2P file sharing
- Multi-user network games
- Streaming stored video clips
- Internet telephone
- Real-time video conference
- Massive parallel computing

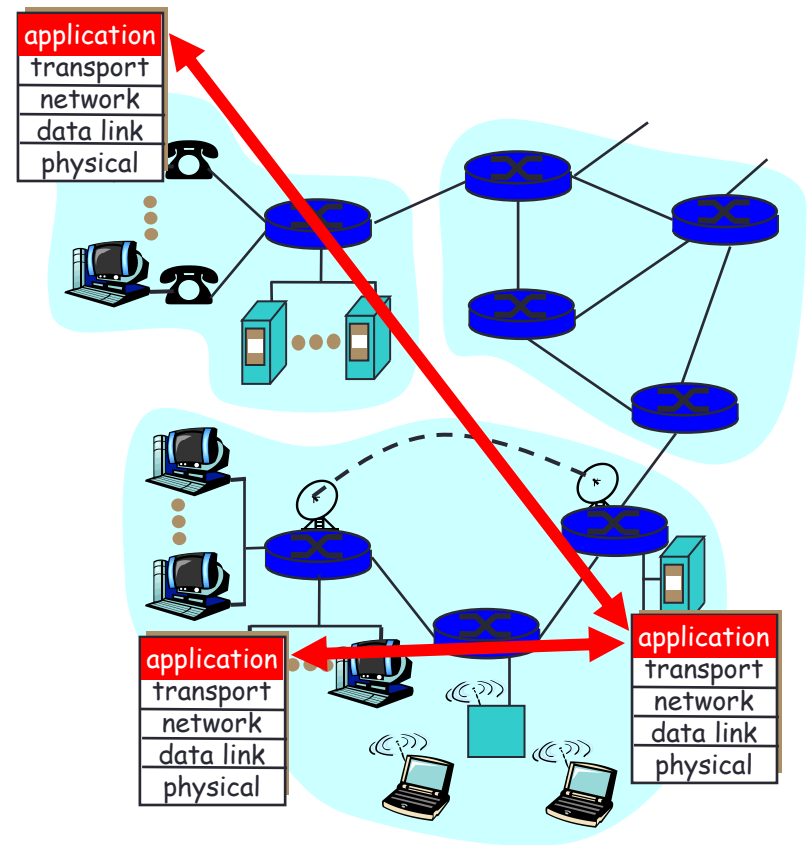
Creating a network app

Write programs that

- run on different end systems and
- communicate over a network.
- e.g., Web: Web server software communicates with browser software

No software written for devices in network core

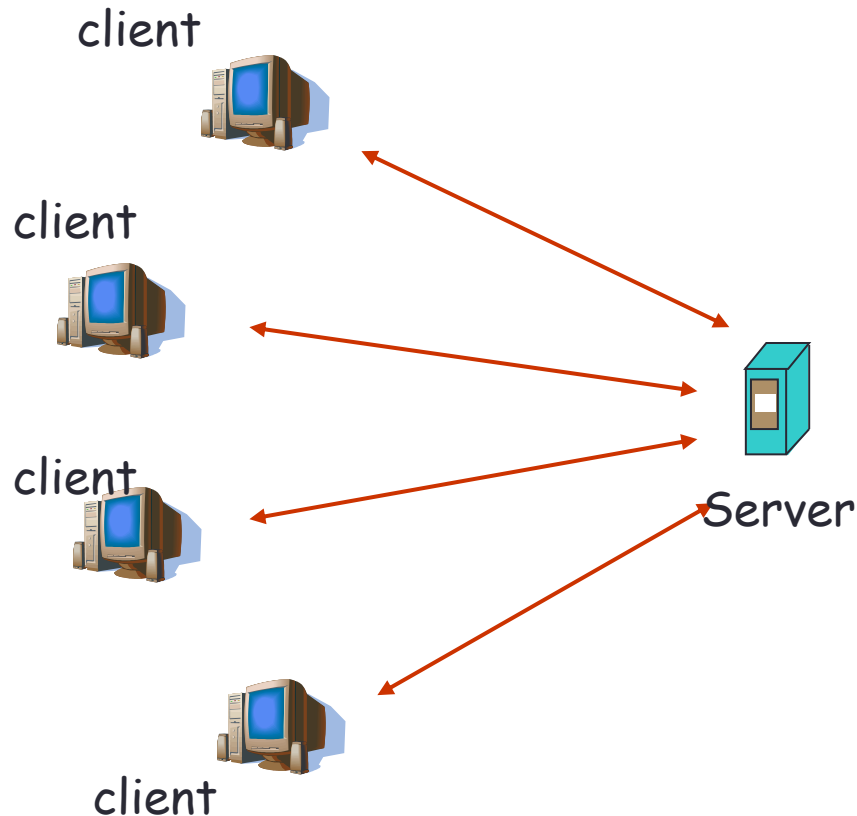
- Network core devices do not function at app layer???



Application architectures

- Client-server
- Peer-to-peer (P2P)
- Hybrid of client-server and P2P

Client-server architecture



clients:

- communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do not communicate directly with each other

server:

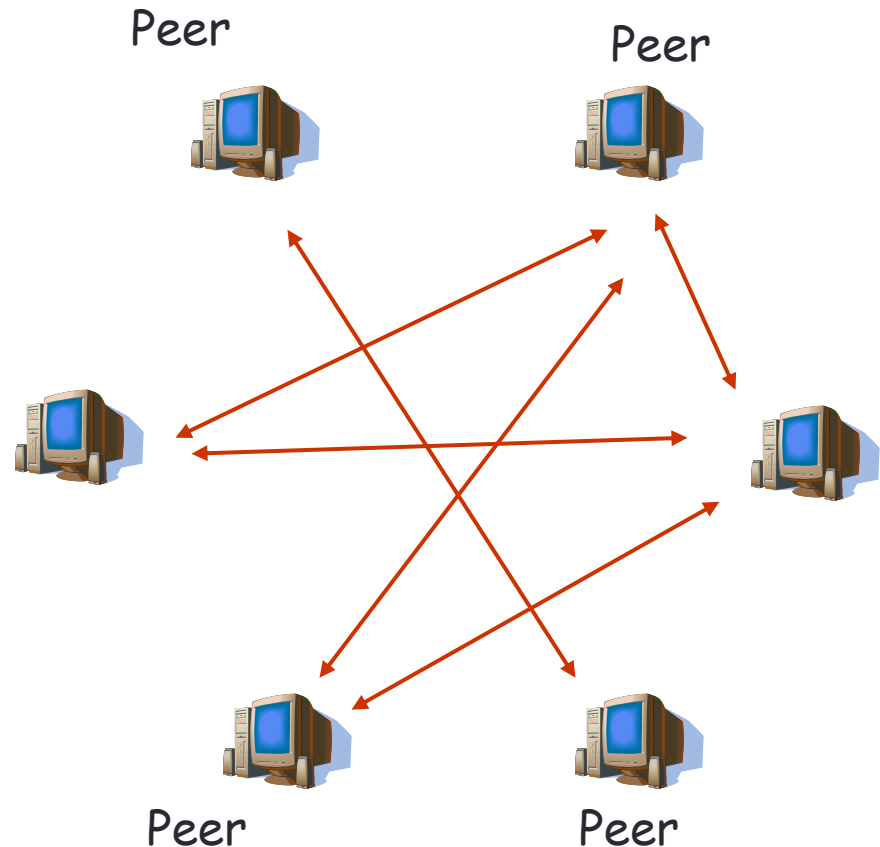
- always-on host
- permanent IP address
- server farms for scaling

Pure P2P architecture

- no always on server
- arbitrary end systems directly communicate
- peers are intermittently connected and change IP addresses
- example: Gnutella

Highly scalable

But difficult to manage



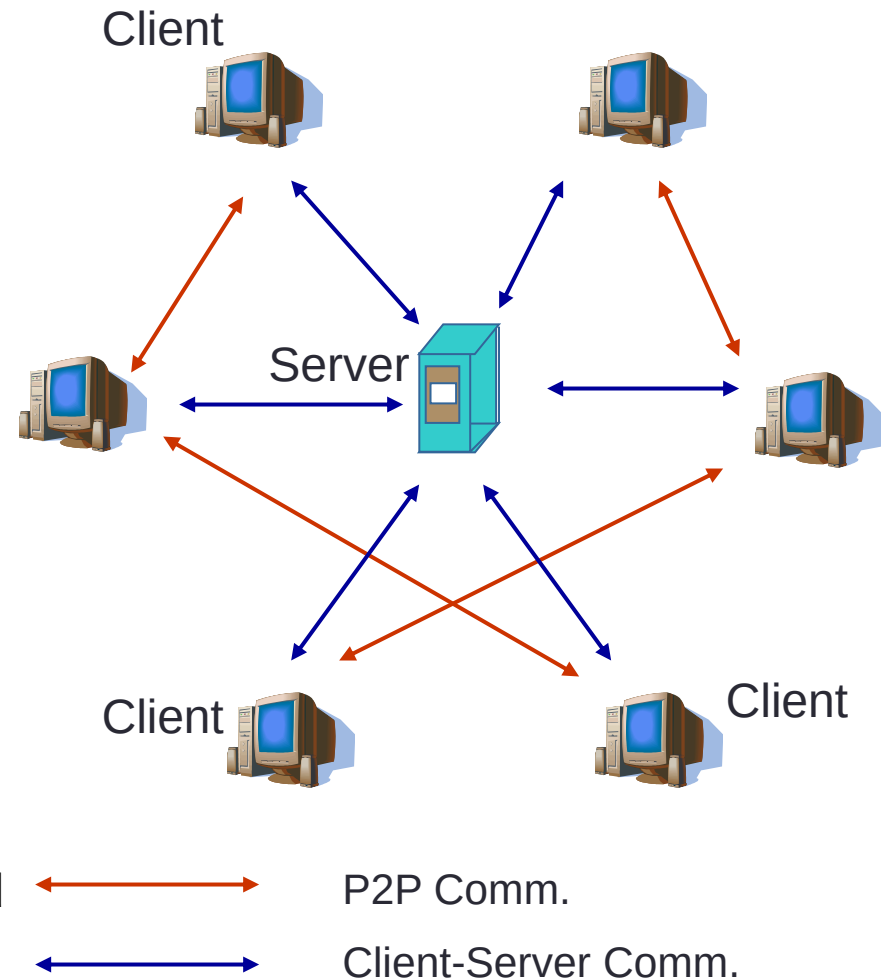
Hybrid of client-server and P2P

BitTorrent

- File transfer P2P
- File search centralized:
 - Peers register content at central server
 - Peers query same central server to locate content

Instant messaging

- Chatting between two users is P2P
- Presence detection/location centralized:
 - User registers its IP address with central server when it comes online
 - User contacts central server to find IP addresses of buddies



Processes communicating

Process: program running within a host.

- within same host, two processes communicate using **inter-process communication** (defined by OS).
- processes in different hosts communicate by exchanging **messages**

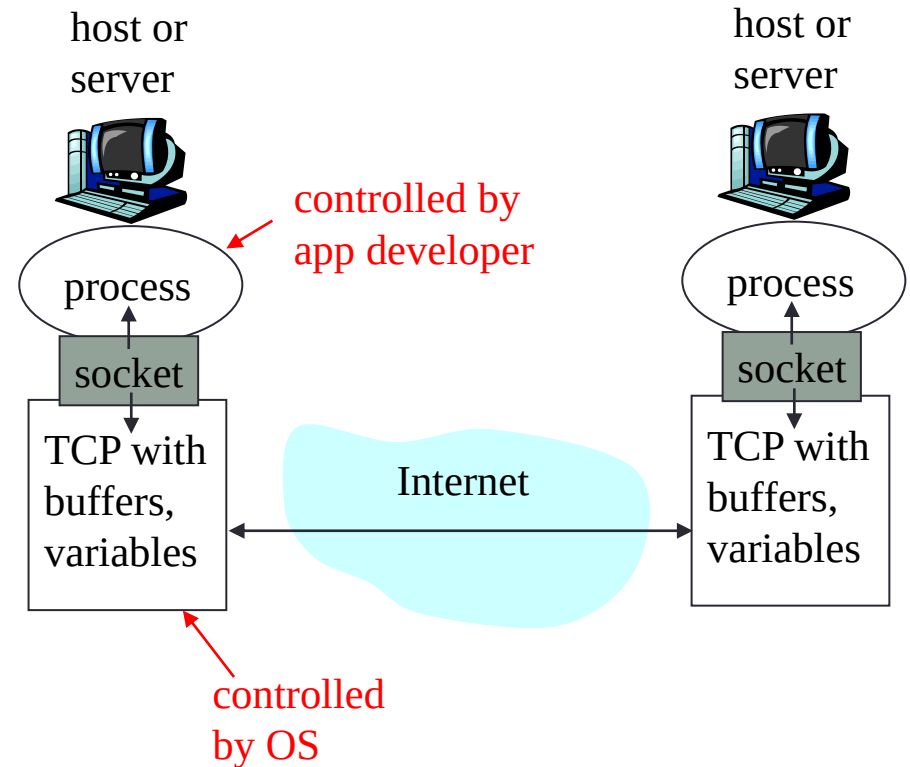
Client process: process that initiates communication

Server process: process that waits to be contacted

- **Note:** applications with P2P architectures have client processes & server processes

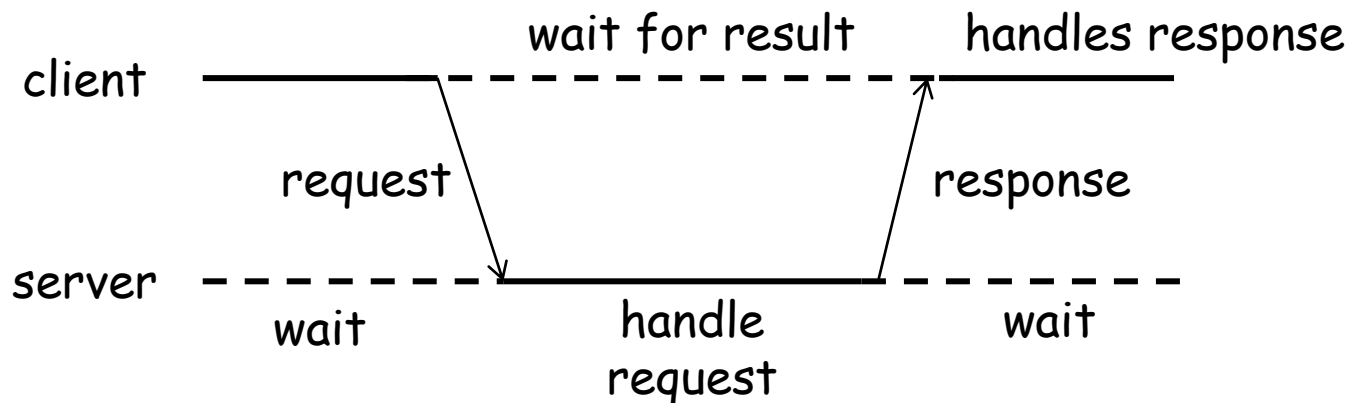
Sockets

- process sends/receives messages to/from its **socket**
- Defined by
 - Port number
 - IP Address } Socket address
- TCP/UDP
- API: (1) choice of transport protocol; (2) ability to fix a few parameters



Processes communicating

- Client process: sends request
- Server process: replies response
- Typically: single server - multiple clients
- The server does not need to know anything about the client
- The client should always know something about the server
 - at least the socket address of the server



App-layer protocol defines

- Types of messages exchanged, e.g, request & response messages
- Syntax of message types: what fields in messages & how fields are delineated
- Semantics of the fields, e.g., meaning of information in fields
- Rules for when and how processes send & respond to messages

What transport service does an app need?

Data loss

- some apps (e.g., audio) can tolerate some loss
- other apps (e.g., file transfer, telnet) require 100% reliable data transfer

Timing

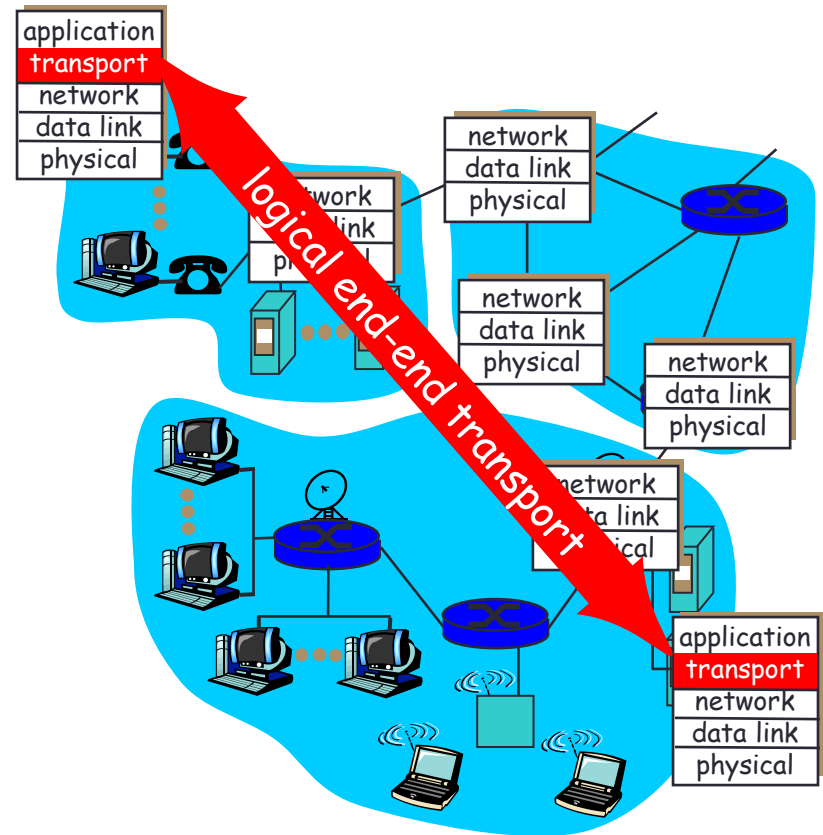
- some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

Bandwidth

- ❑ some apps (e.g., multimedia) require minimum amount of bandwidth to be “effective”
- ❑ other apps (“elastic apps”) make use of whatever bandwidth they get

Transport services and protocols

- provide *logical communication* between app processes running on different hosts
- transport protocols run in end systems
 - send side: breaks app messages into **segments**, passes to network layer
 - rcv side: reassembles segments into messages, passes to app layer
- more than one transport protocol available to apps
 - Internet: TCP and UDP



Internet transport protocols services

TCP service:

- *reliable transport* between sending and receiving process
- *flow control*: sender won't overwhelm receiver
- *congestion control*: throttle sender when network overloaded
- *does not provide*: timing, minimum throughput guarantee, security
- *connection-oriented*: setup required between client and server processes

UDP service:

- *unreliable data transfer* between sending and receiving process
- *does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup,

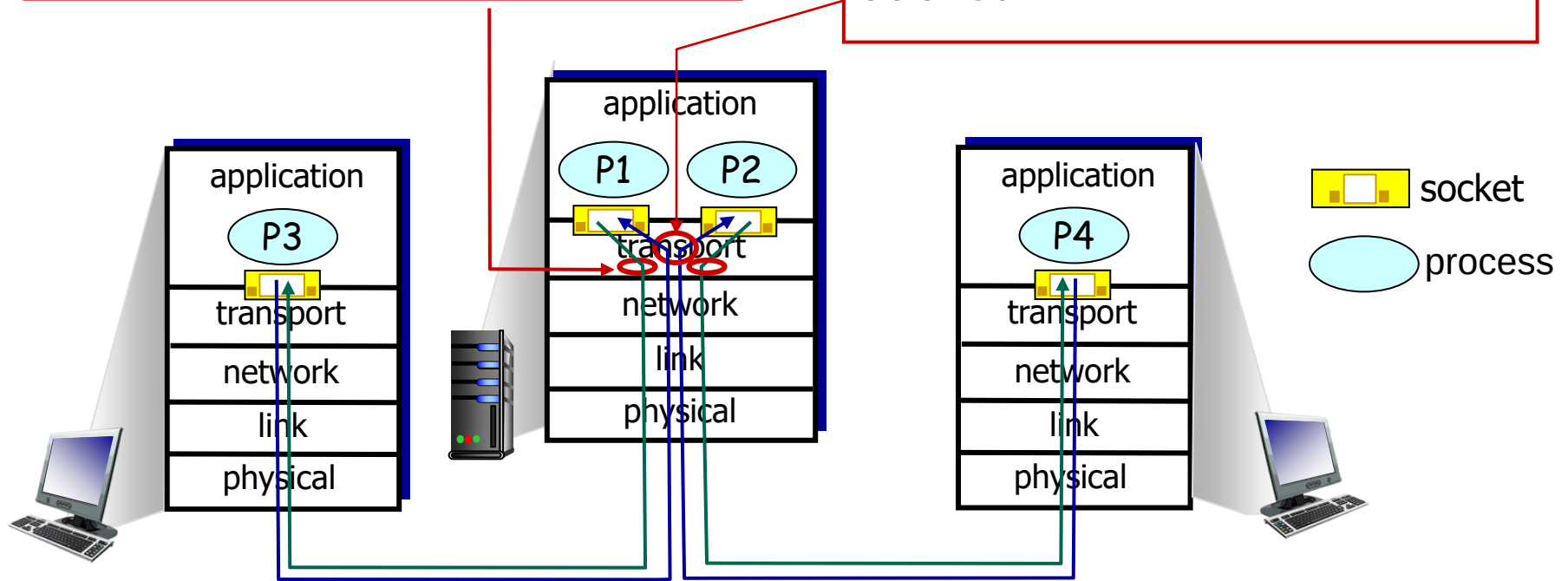
Multiplexing/demultiplexing

multiplexing at sender:

handle data from multiple sockets, add transport header (later used for demultiplexing)

demultiplexing at receiver:

use header info to deliver received segments to correct socket

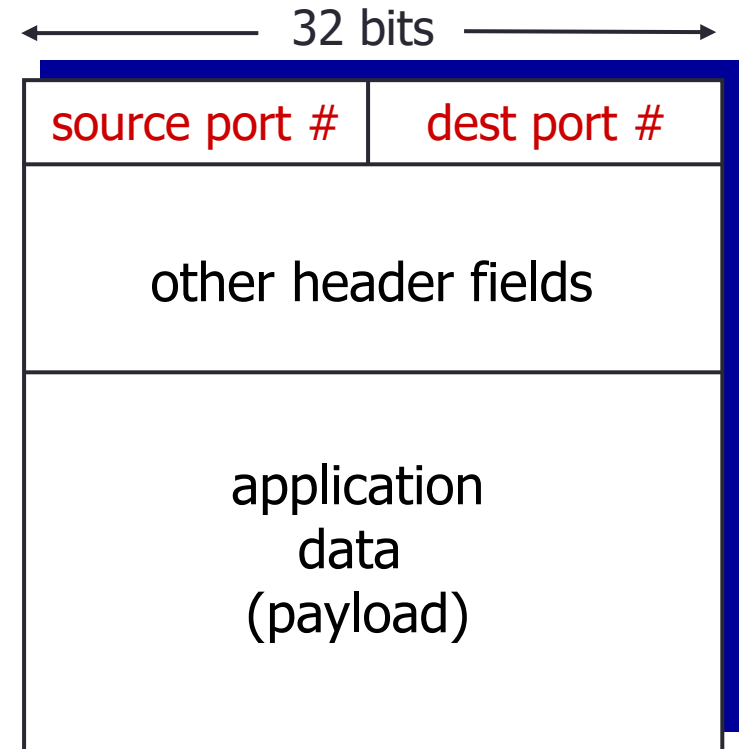


How demultiplexing works

❖ host receives IP datagrams

- each datagram has source IP address, destination IP address
- each datagram carries one transport-layer segment
- each segment has source, destination port number

❖ host uses *IP addresses* & *port numbers* to direct segment to appropriate socket



TCP/UDP segment format

UDP: User Datagram Protocol [RFC 768]

- “no frills,” “bare bones” Internet transport protocol
- “best effort” service, UDP segments may be:
 - lost
 - delivered out of order to app
- *connectionless*:
 - no handshaking between UDP sender, receiver
 - each UDP segment handled independently of others

Why is there a UDP?

- no connection establishment (which can add delay)
- simple: no connection state at sender, receiver
- small segment header
- no congestion control: UDP can blast away as fast as desired

UDP demultiplexing

- Create sockets with port numbers:

```
mySocket = socket(AF_INET,  
    SOCK_DGRAM, 0)
```

- UDP socket identified by two-tuple:

(dest IP address, dest port number)

- When host receives UDP segment:

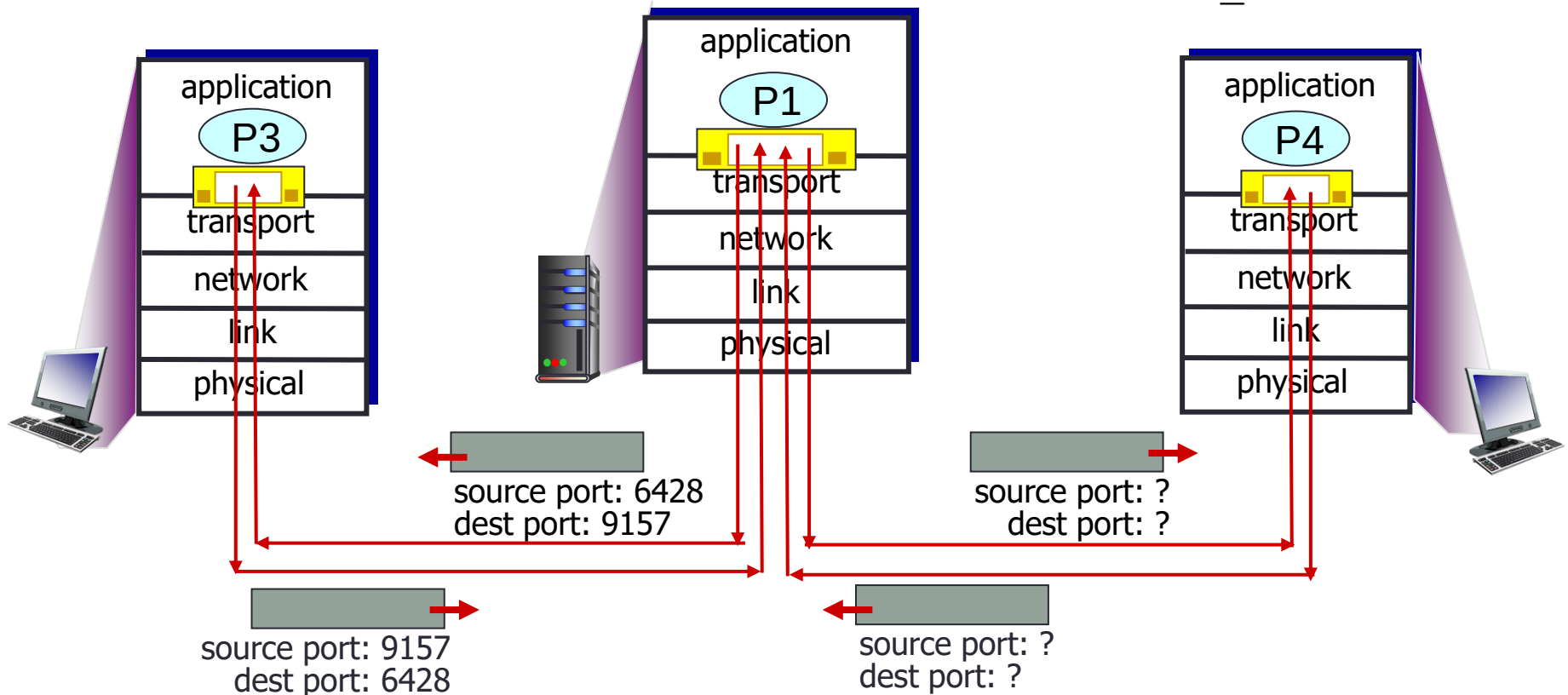
- checks destination port number in segment
- directs UDP segment to socket with that port number
- IP datagrams with different source IP addresses and/or source port numbers directed to same socket

UDP demux

```
mySocket =  
  socket(AF_INET,  
    SOCK_DGRAM, 0);
```

```
serverSocket =  
  socket(AF_INET,  
    SOCK_DGRAM, 0);  
  bind(...)
```

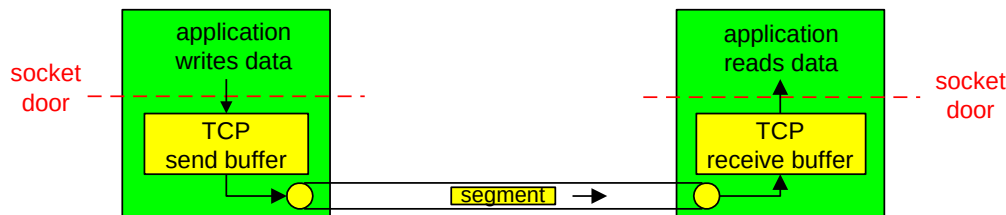
```
mySocket =  
  socket(AF_INET,  
    SOCK_DGRAM, 0);
```



TCP: Overview

RFCs: 793, 1122, 1323, 2018, 2581

- **point-to-point:**
 - one sender, one receiver
- **reliable, in-order *byte stream*:**
 - no “message boundaries”
- **pipelined:**
 - TCP congestion and flow control set window size
- ***send & receive buffers***
- **connection-oriented:**
 - handshaking (exchange of control msgs) init's sender, receiver state before data exchange
- **flow controlled:**
 - sender will not overwhelm receiver
- **full duplex data:**
 - bi-directional data flow in same connection
 - MSS: maximum segment size



TCP Connection Management: Setup

Recall: TCP sender, receiver establish “connection” before exchanging data segments

- initialize TCP variables:
 - seq. #s
 - buffers, flow control info (e.g. **RcvWindow**)
- *client*: connection initiator
 - connect()
- *server*: contacted by client
 - accept()

Three way handshake:

Step 1: client host sends TCP SYN segment to server

- specifies initial seq #
- no data

Step 2: server host receives SYN, replies with SYNACK segment

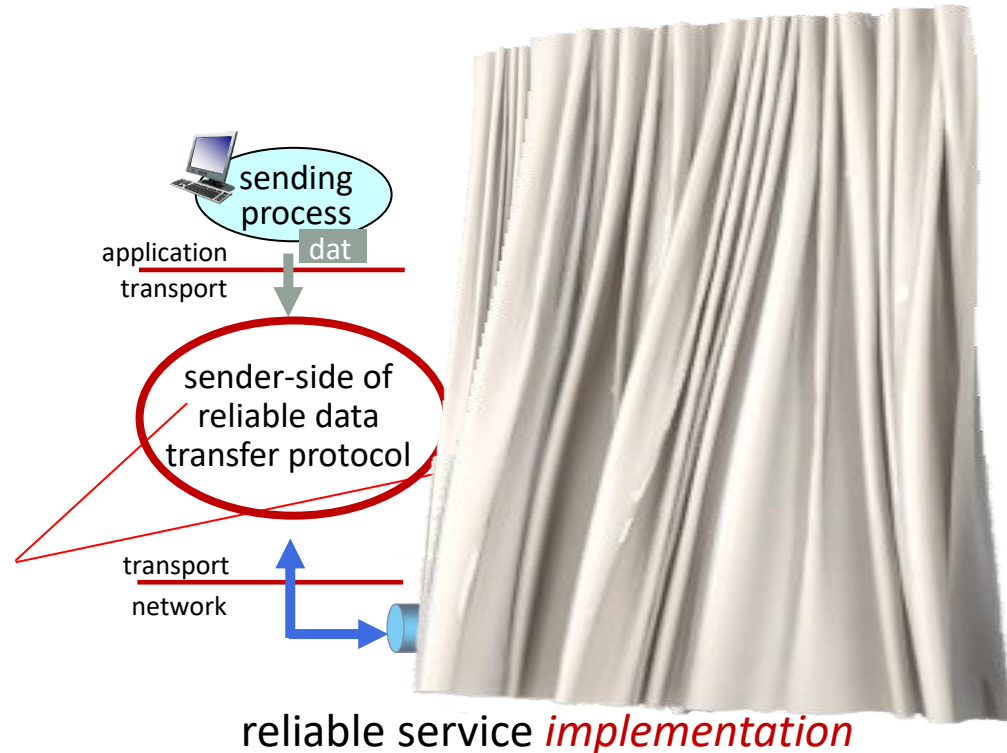
- server allocates buffers
- specifies server initial seq. #

Step 3: client receives SYNACK, replies with ACK segment, which may contain data

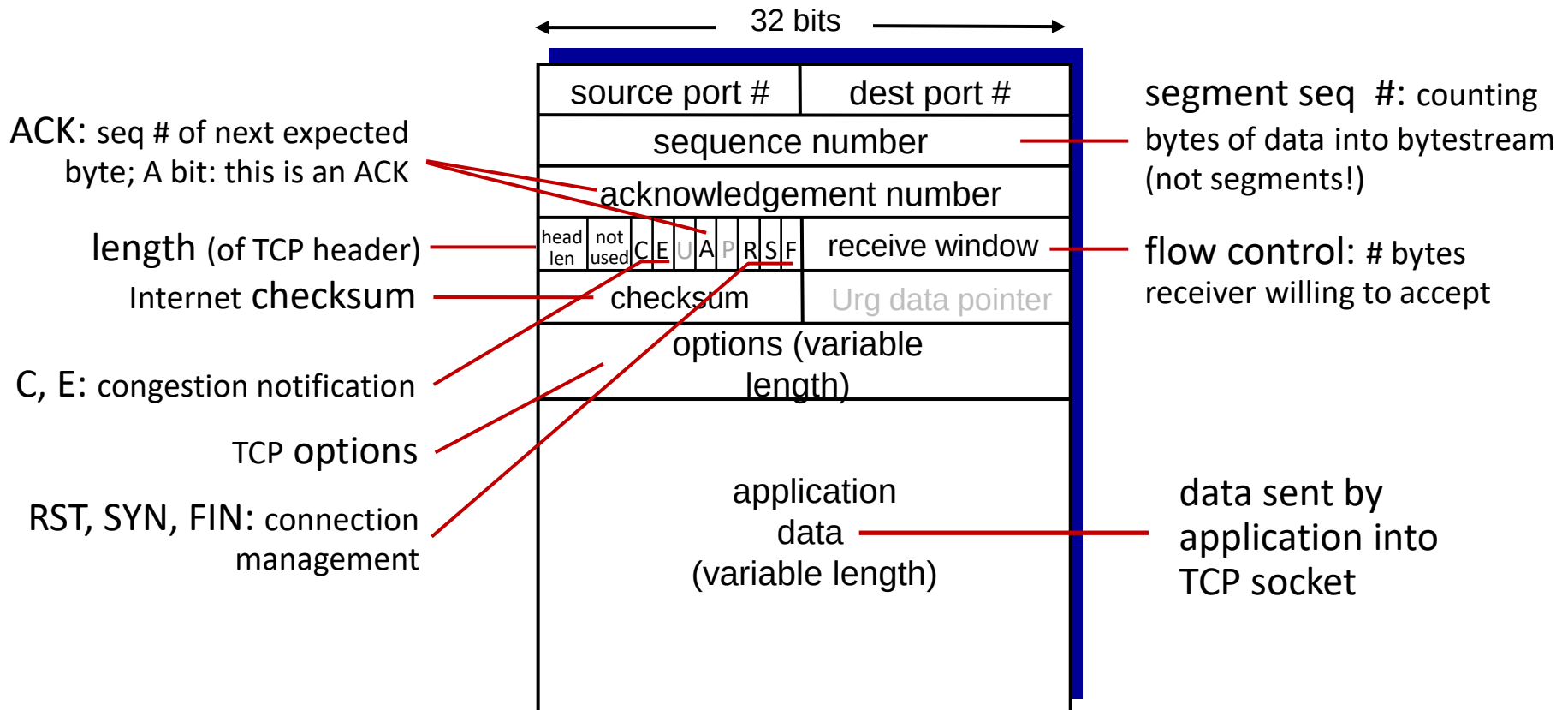
Principles of reliable data transfer

Sender, receiver do *not* know the “state” of each other, e.g., was a message received?

- unless communicated via a message?????



TCP segment structure



TCP sequence numbers, ACKs

Sequence numbers:

- byte stream “number” of first byte in segment’s data

Acknowledgements:

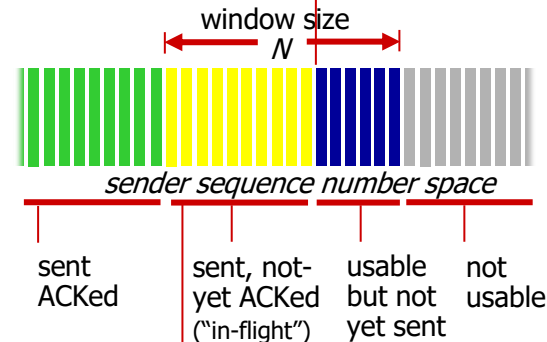
- seq # of next byte expected from other side
- cumulative ACK

Q: how receiver handles out-of-order segments

- A: TCP spec doesn’t say, - up to implementor

outgoing segment from sender

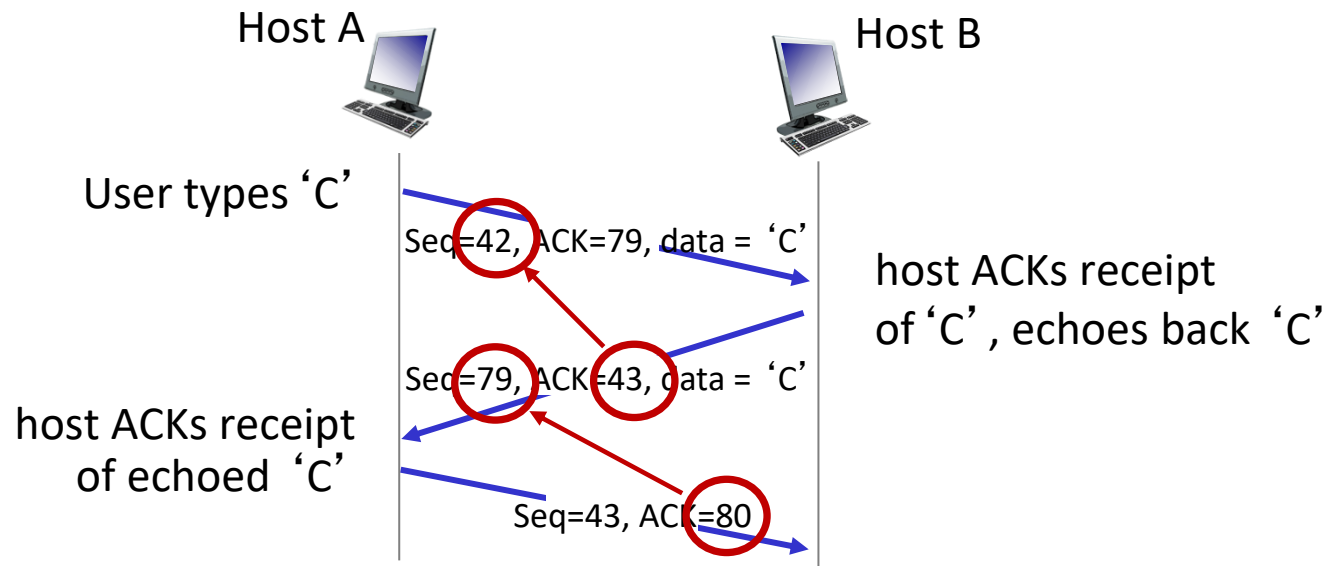
source port #	dest port #
sequence number	
acknowledgement number	
	rwnd
checksum	urg pointer



outgoing segment from receiver

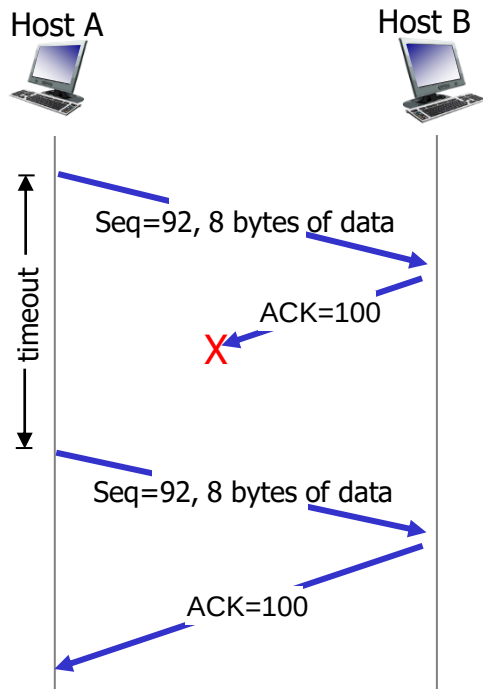
source port #	dest port #
sequence number	
acknowledgement number	
	A
checksum	urg pointer

TCP sequence numbers, ACKs

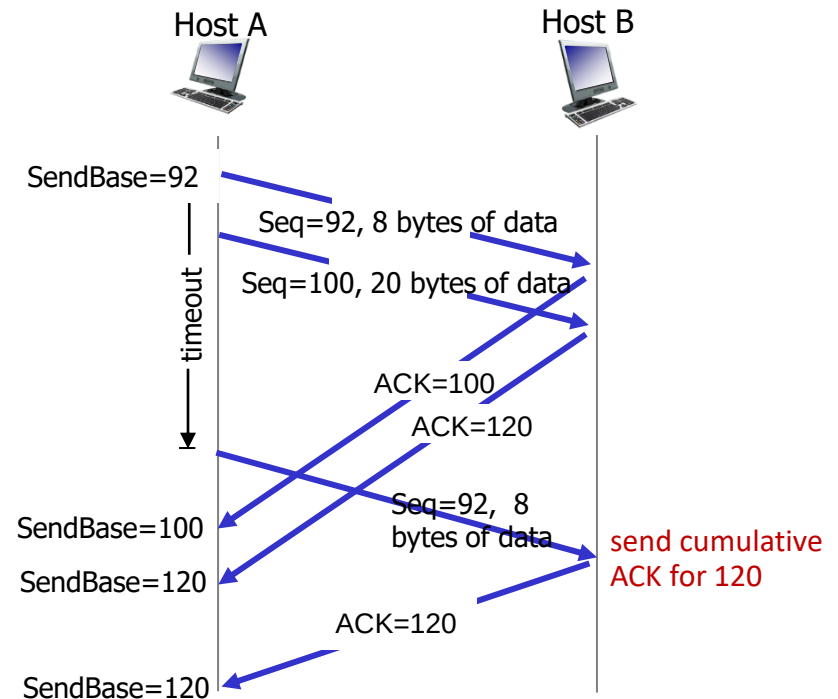


simple telnet scenario

TCP: retransmission scenarios

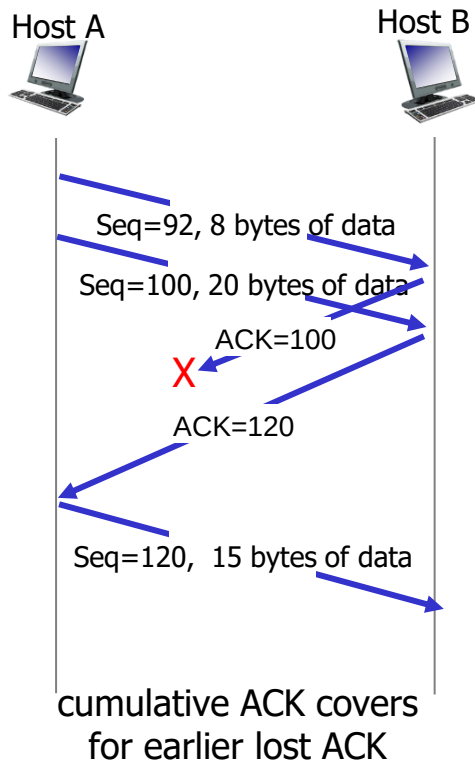


lost ACK scenario



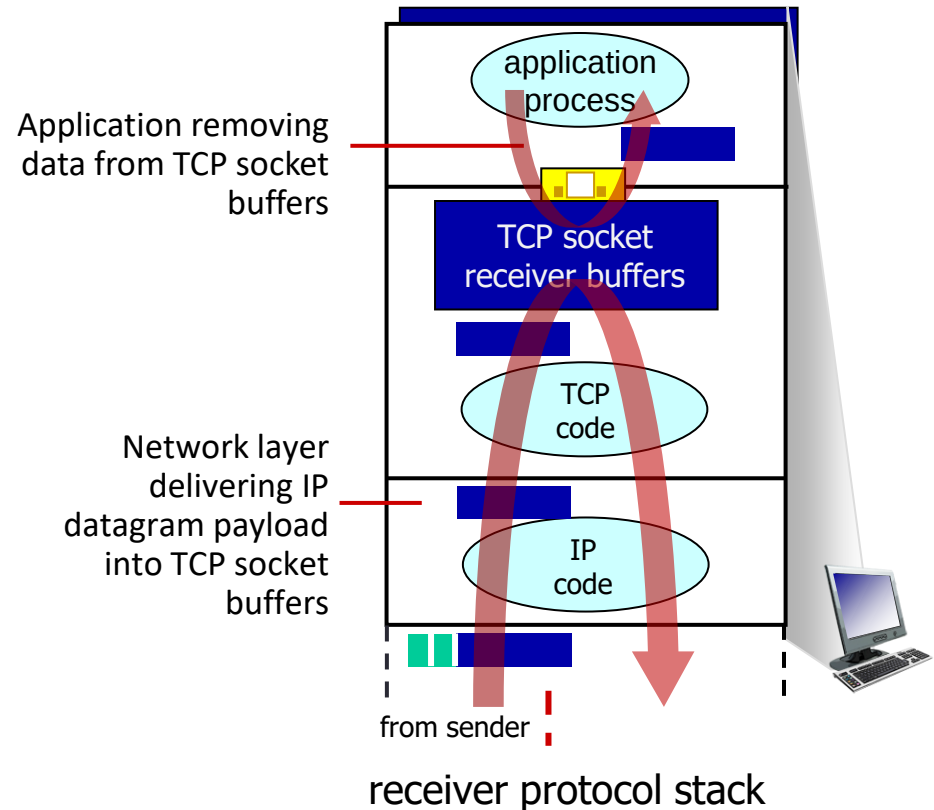
```
premature timeout
```


TCP: retransmission scenarios



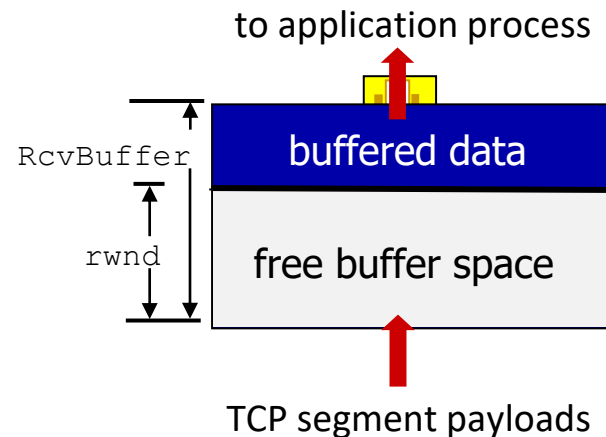
TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?



TCP flow control

- TCP receiver “advertised” free buffer space in `rwnd` field in TCP header
 - `RcvBuffer` size set via socket options (typical default is 4096 bytes)
 - many operating systems auto adjust `RcvBuffer`
- sender limits amount of unACKed (“in-flight”) data to received `rwnd`
- guarantees receive buffer will not overflow



TCP receiver-side buffering

TCP 3-way handshake

Client state

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName, serverPort))
```

SYNSENT

ESTAB

choose init seq num, x
send TCP SYN msg

received SYNACK(x)
indicates server is live;
send ACK for SYNACK;
this segment may contain
client-to-server data



SYNbit=1, Seq=x

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

ACKbit=1, ACKnum=y+1



choose init seq num, y
send TCP SYNACK
msg, acking SYN

received ACK(y)
indicates client is live

Server state

```
serverSocket = socket(AF_INET, SOCK_STREAM)  
serverSocket.bind('', serverPort)  
serverSocket.listen(1)  
connectionSocket, addr = serverSocket.accept()
```

LISTEN

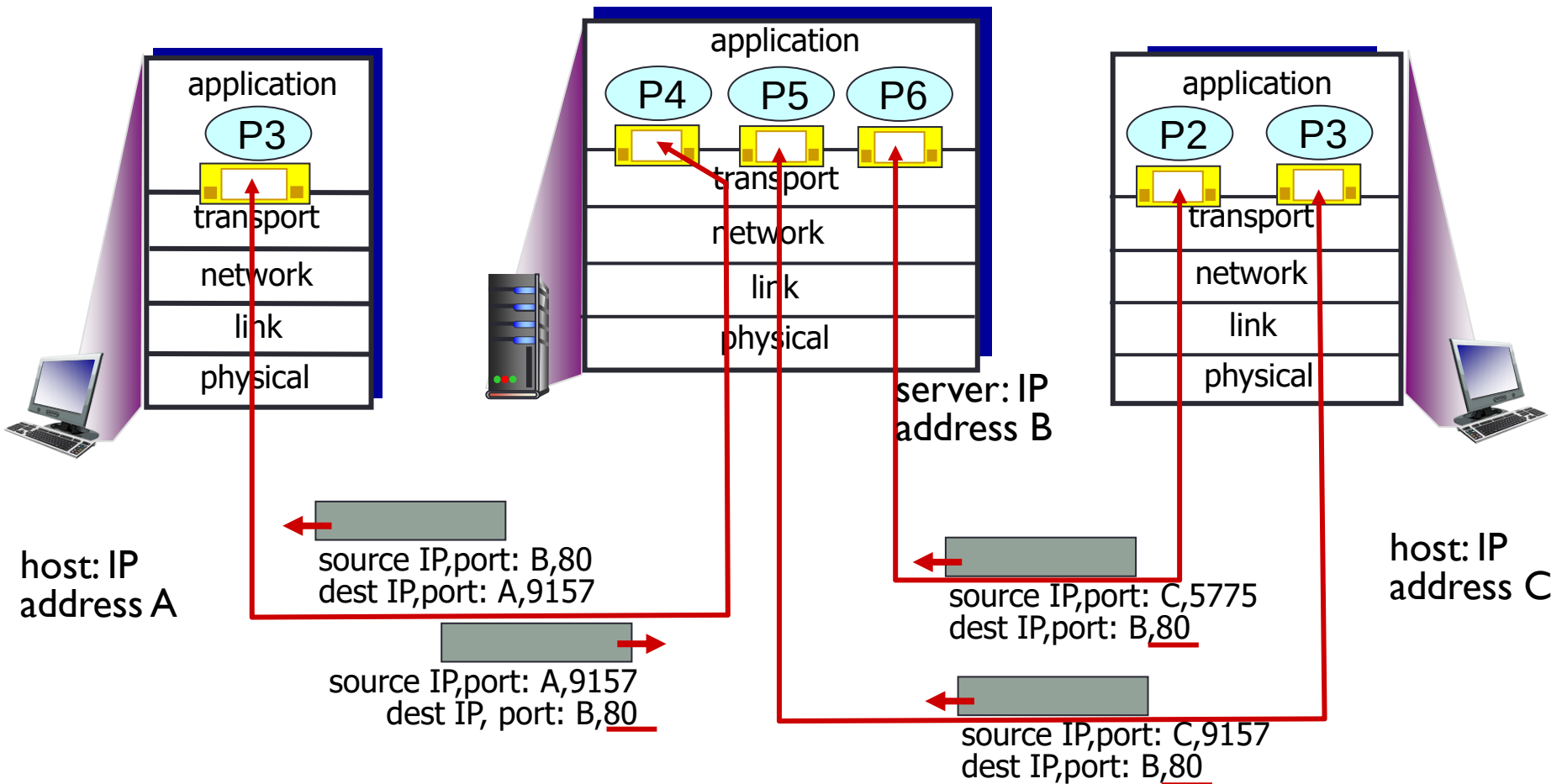
SYN RCVD

ESTAB

Connection-oriented demux

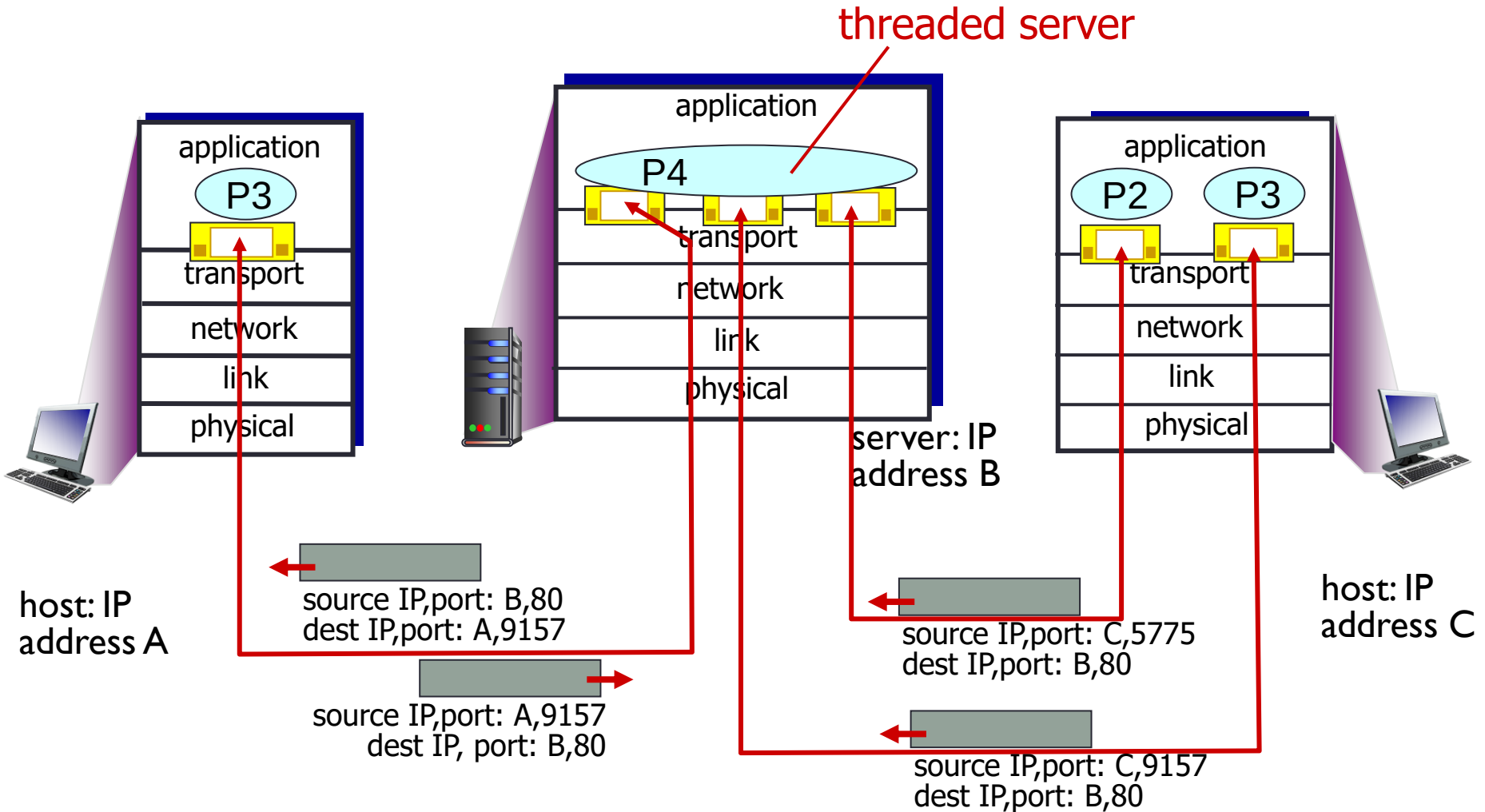
- TCP socket identified by 4-tuple:
 - source IP address
 - source port number
 - dest IP address
 - dest port number
- recv host uses all four values to direct segment to appropriate socket
- Server host may support many simultaneous TCP sockets:
 - each socket identified by its own 4-tuple
- Web servers have different sockets for each connecting client

Connection-oriented demux: example



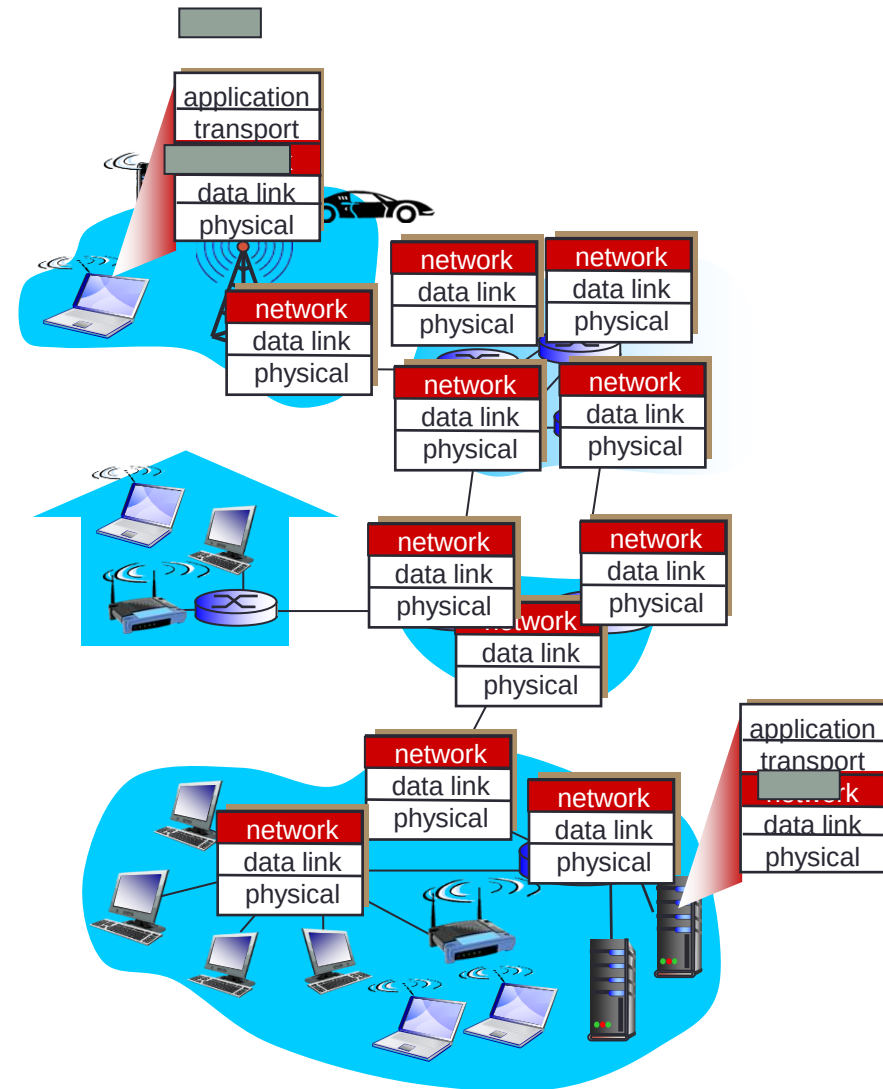
three segments, all destined to IP address: B,
dest port: 80 are demultiplexed to *different* sockets

Connection-oriented demux: example



Network layer

- transport segment from sending to receiving host
- on sending side encapsulates segments into datagrams
- on receiving side, delivers segments to transport layer
- network layer protocols in *every* host, router
- router examines header fields in all IP datagrams passing through it



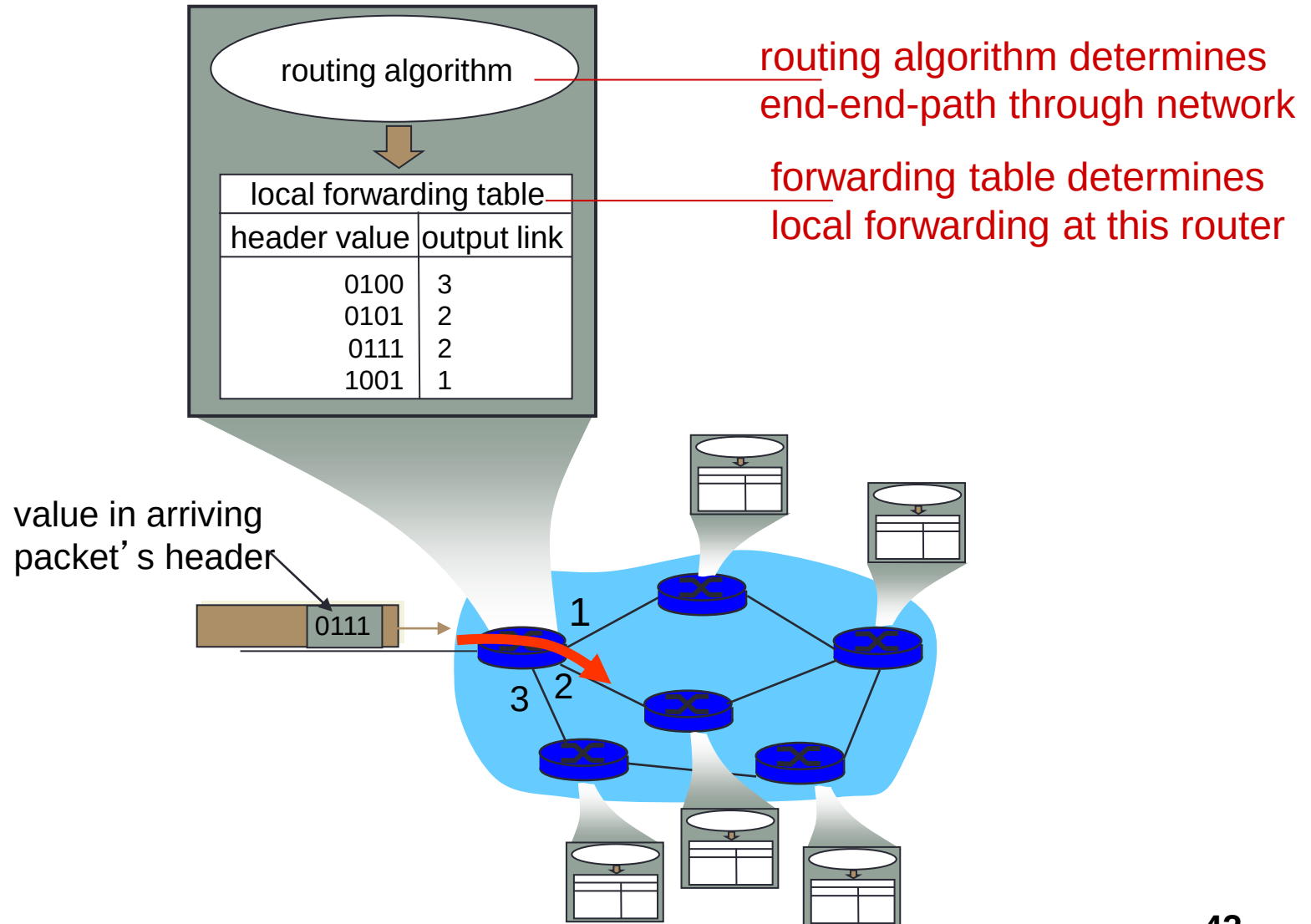
Two key network-layer functions

- *forwarding*: move packets from router's input to appropriate router output
- *routing*: determine route taken by packets from source to dest.
 - *routing algorithms*

analogy:

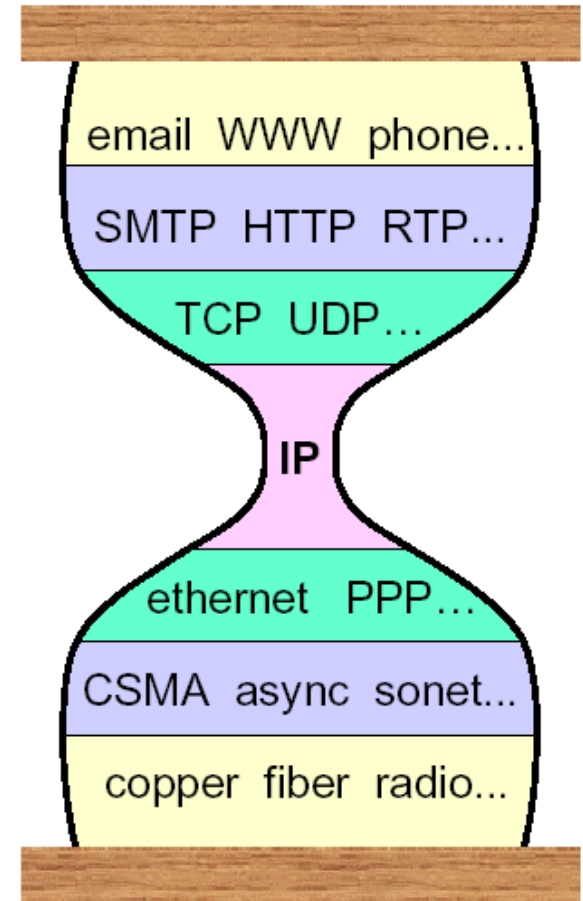
- *routing*: process of planning trip from source to dest
- *forwarding*: process of getting through single interchange

Interplay between routing and forwarding



Why an internet layer?

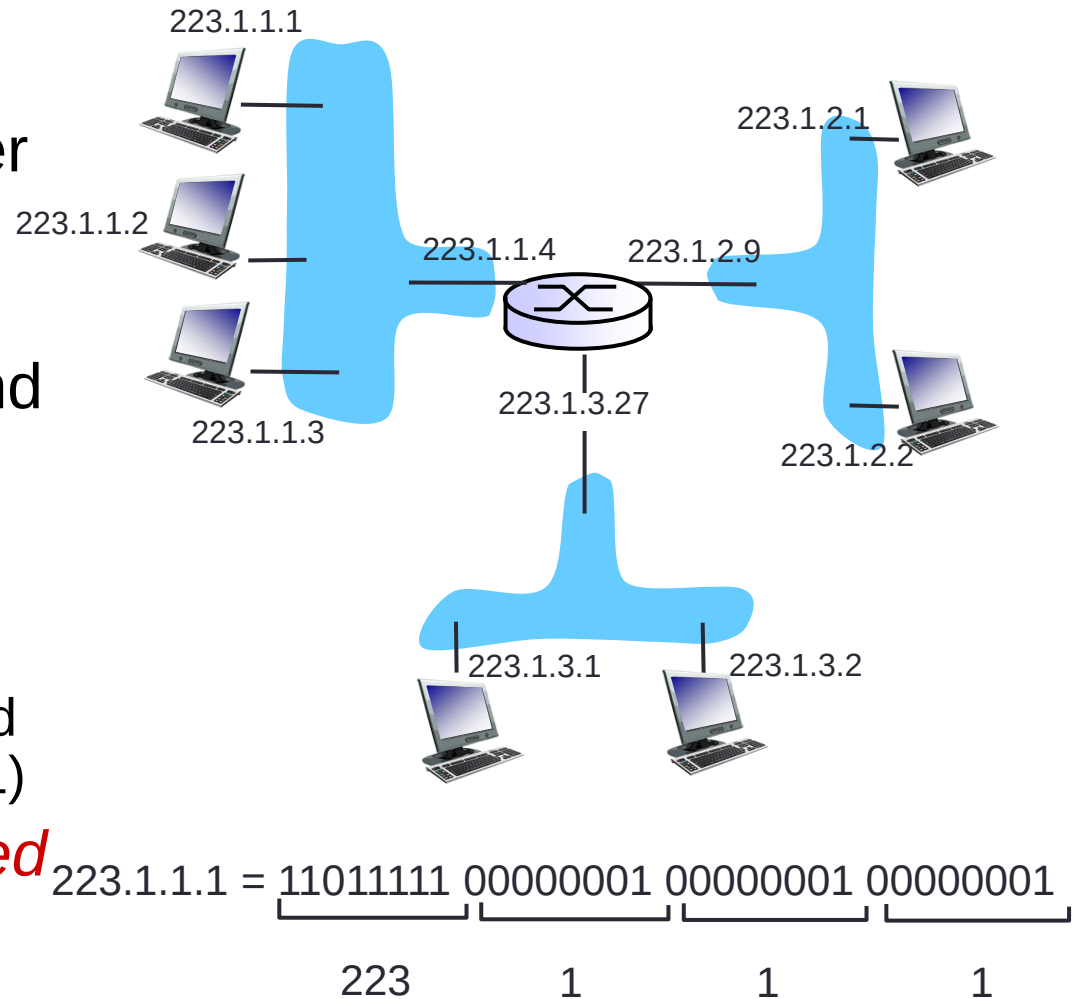
- ❑ Why not one big flat LAN?
 - Different LAN protocols
 - Flat address space not scalable
- ❑ IP provides:
 - Global addressing
 - Scaling to WANs
 - Virtualization of network isolates end-to-end protocols from network details/changes



"hourglass model"
(Steve Deering)

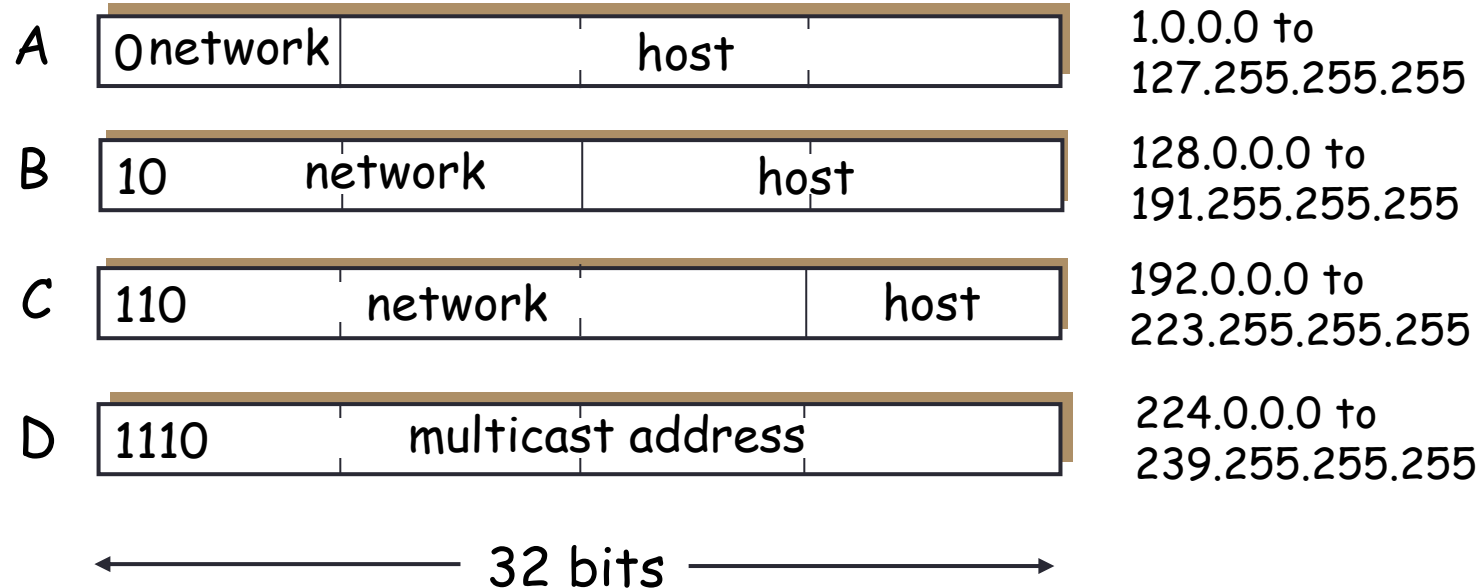
IP addressing: introduction

- **IP address:** 32-bit identifier for host, router interface
- **interface:** connection between host/router and physical link
 - router's typically have multiple interfaces
 - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)
- **IP addresses associated with each interface**



IP addressing: “class-full”

class



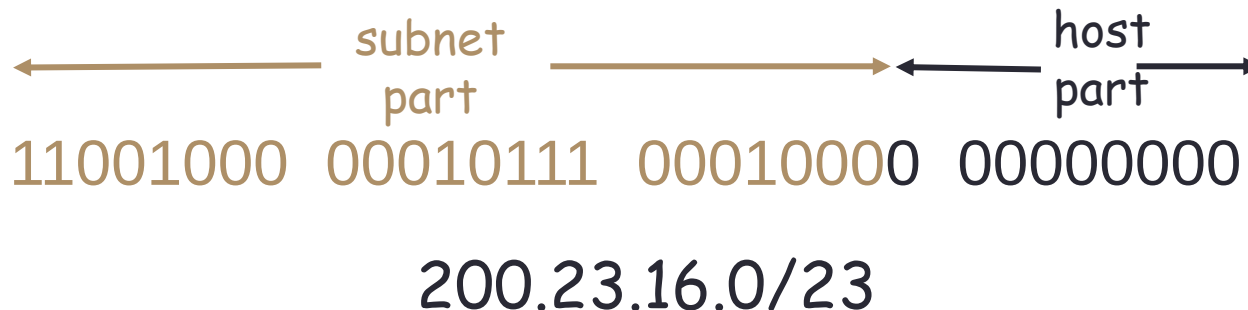
- Class-full addressing:

- inefficient use of address space, address space exhaustion
- e.g., class B net allocated enough addresses for 65K hosts, even if only 2K hosts in that network

IP addressing: “class-less”

CIDR: Classless Inter-Domain Routing

- subnet portion of address of arbitrary length
- address format: **a.b.c.d/x**, where x is # bits in subnet portion of address



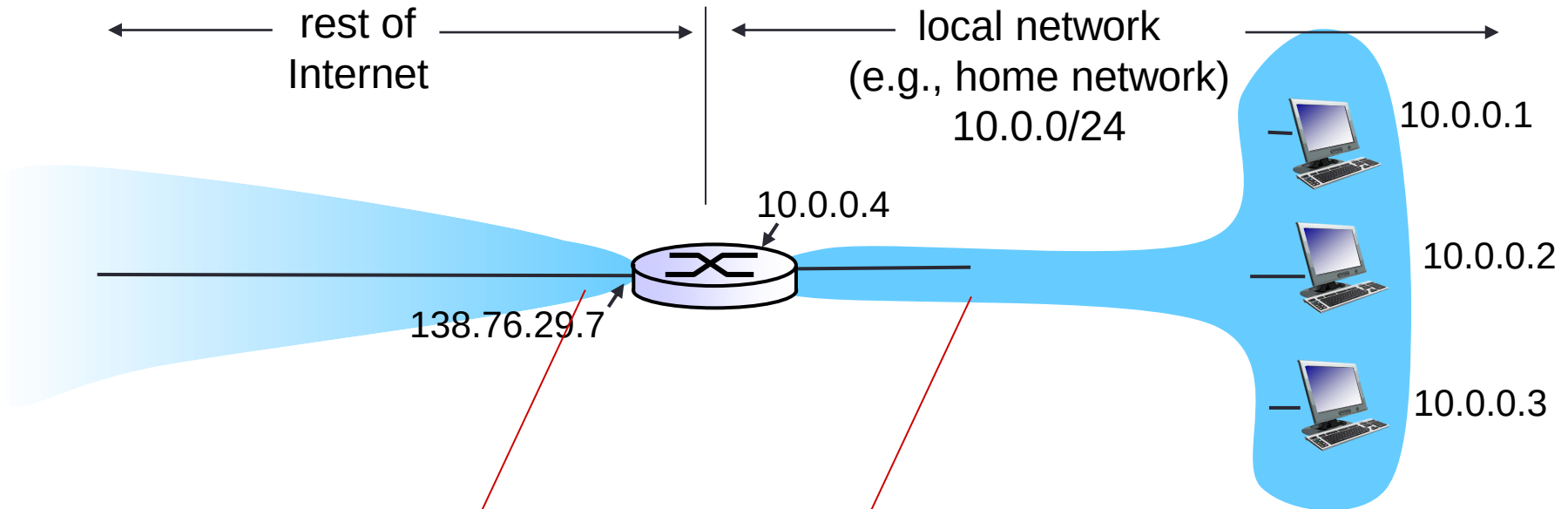
Address Allocation for Private Internets

- RFC1918

Private address	10.0.0.0/8
	172.16.0.0/16 → 172.31.0.0/16
	192.168.0.0/24 → 192.168.255.0 /24
Loopback address	127.0.0.0 /8
Multicast address	224.0.0.0
	~ 239.255.255.255

- Link local address: 169.254.0.0/16

NAT: network address translation



all datagrams *leaving* local network have *same* single source NAT IP address: 138.76.29.7, different source port numbers

datagrams with source or destination in this network have 10.0.0/24 address for source, destination (as usual)

NAT: network address translation

motivation: local network uses just one IP address as far as outside world is concerned:

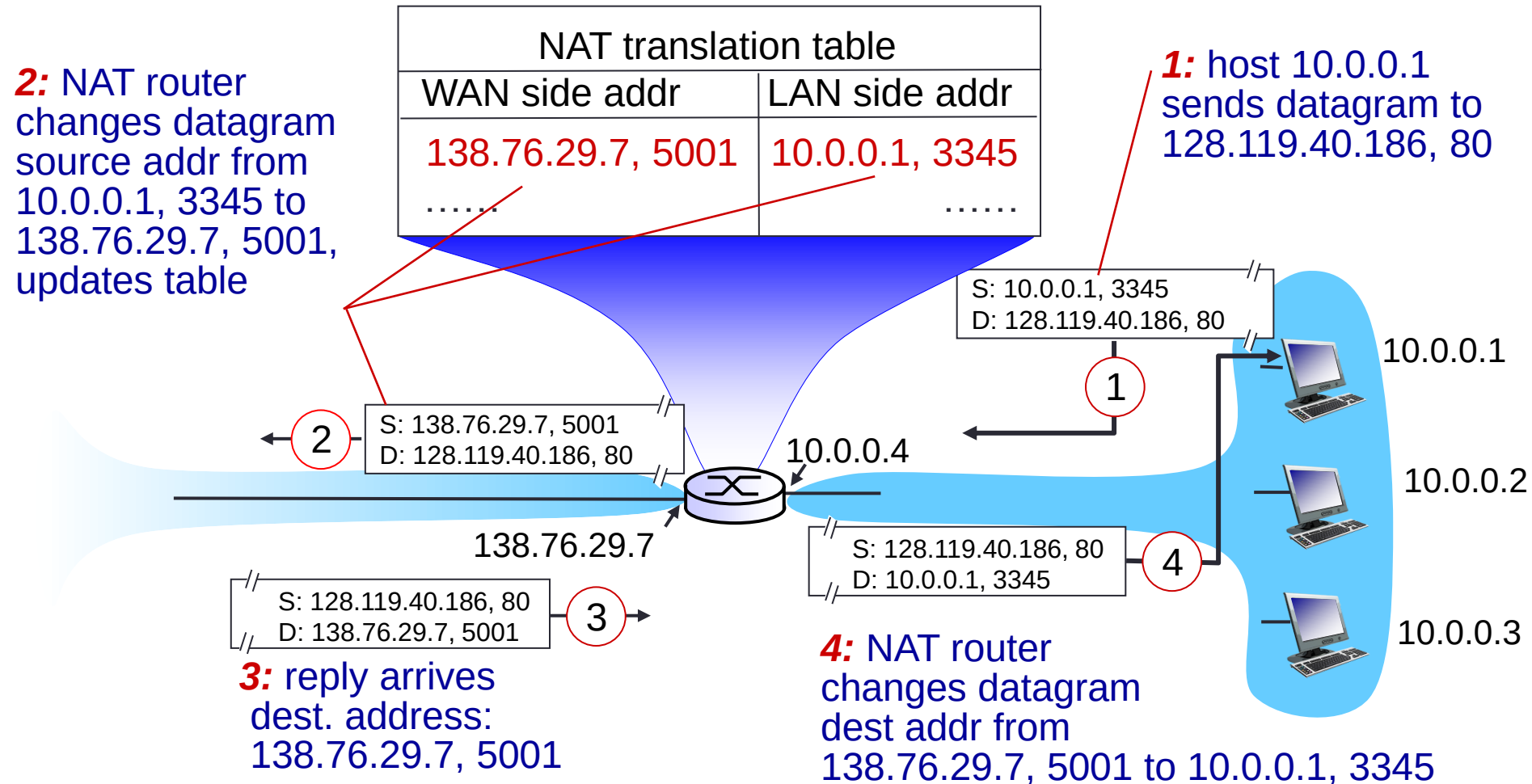
- range of addresses not needed from ISP: just one IP address for all devices
- can change addresses of devices in local network without notifying outside world
- can change ISP without changing addresses of devices in local network
- devices inside local net not explicitly addressable, visible by outside world (a security plus)

NAT: network address translation

implementation: NAT router must:

- *outgoing datagrams: replace* (source IP address, port #) of every outgoing datagram to (NAT IP address, new port #)
... remote clients/servers will respond using (NAT IP address, new port #) as destination addr
- *remember (in NAT translation table)* every (source IP address, port #) to (NAT IP address, new port #) translation pair
- *incoming datagrams: replace* (NAT IP address, new port #) in dest fields of every incoming datagram with corresponding (source IP address, port #) stored in NAT table

NAT: network address translation

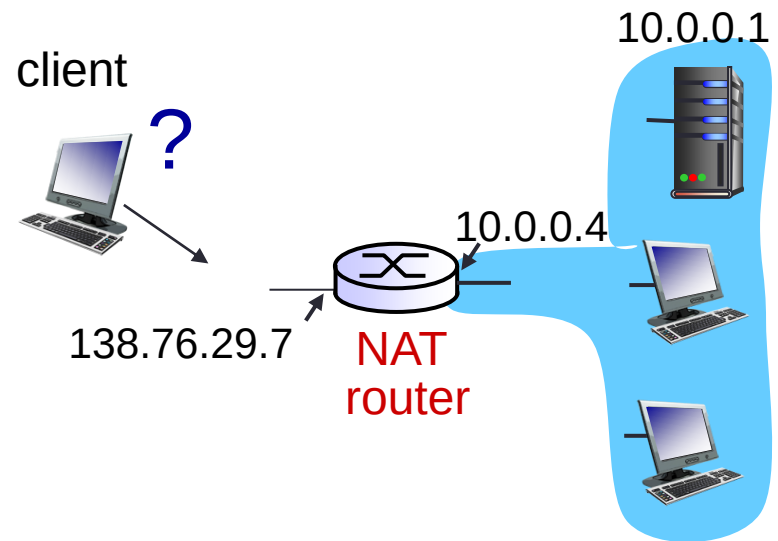


NAT: network address translation

- 16-bit port-number field:
 - 60,000 simultaneous connections with a single LAN-side address!
- NAT is controversial:
 - routers should only process up to layer 3
 - violates end-to-end argument
 - NAT possibility must be taken into account by app designers, e.g., P2P applications
 - address shortage should instead be solved by IPv6

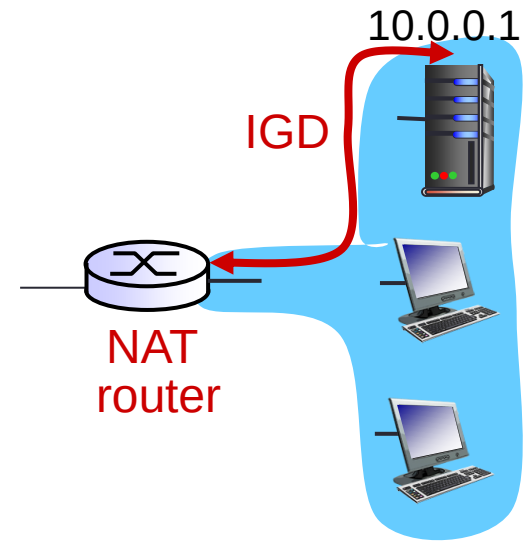
NAT traversal problem

- client wants to connect to server with address 10.0.0.1
 - server address 10.0.0.1 local to LAN (client can't use it as destination addr)
 - only one externally visible NATed address: 138.76.29.7
- **solution1:** statically configure NAT to forward incoming connection requests at given port to server
 - e.g., (123.76.29.7, port 2500) always forwarded to 10.0.0.1 port 25000



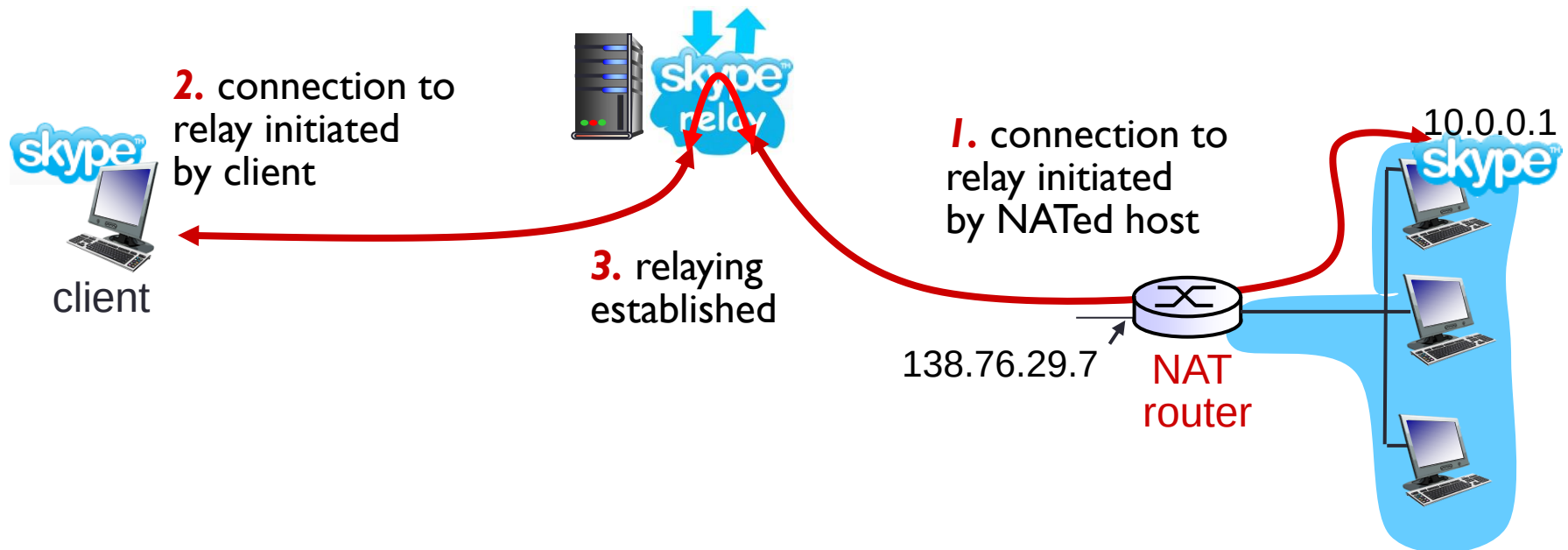
NAT traversal problem

- *solution 2*: Universal Plug and Play (UPnP) Internet Gateway Device (IGD) Protocol. Allows NATed host to:
 - ❖ learn public IP address (138.76.29.7)
 - ❖ add/remove port mappings (with lease times)
- i.e., automate static NAT port map configuration



NAT traversal problem

- **solution 3:** relaying (used in Skype)
 - NATed client establishes connection to relay
 - external client connects to relay
 - relay bridges packets between to connections



NAT traversal problem

- *solution 4*: NAT hole punching. Example: STUN protocol

